

Our function will simply generate a list of some given length, run our insertion sort code, and return the final value it finds for `stepcounter`.

```
def check_steps(size_of_cabinet):
    cabinet = [int(1000 * random.random()) for i in range(size_of_cabinet)]
    global stepcounter
    stepcounter = 0
    sortedcabinet = insertion_sort(cabinet)
    return(stepcounter)
```

Let's create a list of all numbers between 1 and 100 and check the number of steps required to sort lists of each length.

```
random.seed(5040)
xs = list(range(1,100))
ys = [check_steps(x) for x in xs]
print(ys)
```

In this code, we start by calling the `random.seed()` function. This is not necessary but will ensure that you see the same results as those printed here if you run the same code. You can see that we define sets of values for `x`, stored in `xs`, and a set of values for `y`, stored in `ys`. The `x` values are simply the numbers between 1 and 100, and the `y` values are the number of steps required to sort randomly generated lists of each size corresponding to each `x`. If you look at the output, you can see how many steps insertion sort took to sort randomly generated lists of lengths 1, 2, 3 . . . , all the way to 99. We can plot the relationship between list length and sorting steps as follows. We'll import `matplotlib.pyplot` in order to accomplish the plotting.

```
import matplotlib.pyplot as plt
plt.plot(xs,ys)
plt.title('Steps Required for Insertion Sort for Random Cabinets')
plt.xlabel('Number of Files in Random Cabinet')
plt.ylabel('Steps Required to Sort Cabinet by Insertion Sort')
plt.show()
```

[Figure 4-1](#) shows the output. You can see that the output curve is a little jagged—sometimes a longer list will be sorted in fewer steps than will a shorter list. The reason for this is that we generated every list randomly. Occasionally our random list generation code will create a list that's easy for insertion sort to deal with quickly (because it's already partially sorted),